AI Configuration

> **Redacted for public sharing.** Employer-specific identifiers have been masked: production-adjacent hostnames as `<company-host>`, the internal Perl package as `<COMPANY>`, its env symbol as `<COMPANY_ENV>`, the dev-DB hostname prefix as `<dev-db>`, and the dev-env flag as `<DEV_FLAG>`. Everything else is verbatim.

Contents

- Overview
- Directory structure
- Loading structure
- Phrase commands
- Hooks
- Scripts
- Settings
- Rules
- How rules load
- git
- perl
- dev-guardrail
- Skills
- Create structured plans with task tracking
- Follow a setup doc step-by-step on a clean target
- Sync changes to a remote host for testing
- Debate a decision with another looping LLM
- Context
- Instructions
- Ansible Integration
- Manual setup on a foreign machine
- Backing up Claude
- Appendix: bundled resource files

Overview

This directory contains rules, skills, hooks, context, settings, and instructions that configure Claude Code's behavior.

Directory structure

On new systems, symlink the following into `~/.claude/`:

Repo path	What	`~/.claude/` symlink	Symlink target
`instructions/CLAUDE.md`	Global system prompt loaded into every Claude session	`~/.claude/CLAUDE.md`	`$HOME/repos/config/AI/instructions/CLAUDE.md`
`rules/`	Coding & git conventions, injected by hooks only when relevant	`~/.claude/rules/`	`$HOME/repos/config/AI/rules/`
`skills/`	Reusable multi-step task workflows, invoked on demand	`~/.claude/skills/`	`$HOME/repos/config/AI/skills/`
`hooks/`	Event-driven scripts that inject context or gate tool use	`~/.claude/hooks/`	`$HOME/repos/config/AI/hooks/`

Repo path	What	`~/.claude/` symlink	Symlink target
`context/`	Context fragments injected only when a hook's trigger fires	`~/.claude/context/`	`$HOME/repos/config/AI/context/`
`settings.json`	Hook definitions plus model, theme, and permission preferences	`~/.claude/settings.json`	`$HOME/repos/config/AI/settings.json`

Loading structure

Everything is conditional — **zero tokens until its trigger fires** (the lone exception is `CLAUDE.md`, always present but currently an empty stub):

Piece	Trigger	Token use
`instructions/CLAUDE.md`	Always — the base prompt for every session	Every session (~0, empty)
`rules/git.md`	Working directory is under `~/repos/` or `~/src/git/` — `SessionStart` hook	0 until in a repo
`rules/perl.md`	You `Read` a Perl file matching its `paths:` patterns — `PreToolUse` hook	0 until you open Perl
`rules/dev-guardrail.md`	Mechanical gate, not a context load — a `PreToolUse(Bash)` hook arms when cwd is under `~/src/git` **and** the host matches its `hosts:` patterns (), then blocks Bash unless dev is confirmed	0 always (never enters context; only a block emits text)
magic-phrase skills & context	An exact phrase in your prompt — see Phrase commands	0 until you type it
other `skills/*.md`	Only when you ask Claude to read the file (manual)	0 until loaded

Phrase commands

Type any of these exact, case-sensitive phrases to summon a skill or framework on demand — **zero idle cost**, dispatched by `inject-on-phrase.sh` (one table row each). They fire whether the phrase ends your message or has text after it.

Phrase	What it does	Where	Example you type	Result
`RECOMMEND TO ME`	Structured recommendation framework	Only in a repo	`RECOMMEND TO ME whether to cache the parsed config in memory`	An analysis through these angles (author intent, least astonishment, current behaviour, with-docs, without-docs) ending in one synthesized verdict
`CREATE A PLAN`	create-plan skill (`.md`-aware naming)	Anywhere	`CREATE A PLAN auth-refactor`	Scaffolds `plans/auth-refactor.md` with a validation table, execution/maintenance rules, and a `NEXT ACTION` line
`PREPARE DEBATE`	debate skill	Anywhere	`PREPARE DEBATE should we keep the trailing-space triggers? 30s`	Creates a `/tmp` channel + a bootstrap prompt to paste into a challenger AI, then the two AIs argue to a verdict

Hooks

Hooks are event-driven automation that fire at specific points in Claude Code's lifecycle. They let you inject context, run validation, or gate tool use — without Claude needing to remember to do it. Hooks are defined in `settings.json` and run as shell commands (see Scripts); their stdout is captured and (for `additionalContext` hooks) injected into the next turn's context.

See the Claude hooks documentation.

Key lifecycle events:

- **`SessionStart`** — fires once when a session begins. Use for conditional context injection that depends on the working directory (e.g. repo-specific rules).
- **`UserPromptSubmit`** — fires when you submit a prompt, *before* it's processed. The hook receives the prompt text and cwd, and can inject context or block the prompt. Use it to gate on what you actually typed (e.g. a magic-word trigger).
- **`PreToolUse`** — fires *before* a tool executes. You can inspect the tool input, inject context about the file being operated on, or even block the tool by exiting non-zero.
- **`PostToolUse`** — fires *after* a tool completes. Use for logging, notifications, or post-processing output.
- **`Stop`** — fires when the session ends. Use for cleanup or persistent state writes.

Hooks are configured in `settings.json`, symlinked to `~/.claude/settings.json`.

Hook	Event	Purpose	Token use
Git rules	`SessionStart`	Injects `git.md` when the working directory is under `~/repos/` or `~/src/git/`	0 until in a repo
Perl rules	`PreToolUse` on `Read`	Runs `inject-perl-rules.sh` before every file read; on match, injects `perl.md`	0 until you open Perl
Dev guardrail	`PreToolUse` on `Bash`	Runs `guard-dev-env.sh`; arms when cwd is under `~/src/git` **and** the host matches `dev-guardrail.md`'s `hosts:` patterns (), then blocks Bash unless dev is confirmed (`<DEV_FLAG>` + `<COMPANY_ENV>` + MySQL `@@hostname`). `<DEV_FLAG>` re-checked every call; only the DB check is memoized (per-session, 5-min TTL). Fails closed	0 on allow (nothing injected); only a block emits text
Phrase commands	`UserPromptSubmit`	`inject-on-phrase.sh` injects the mapped skill/fragment when the prompt contains one of its exact trigger phrases — see Phrase commands	0 until you type a phrase

The hook definitions in `settings.json` (the file also sets `model`, `theme`, and `skipAutoPermissionPrompt`):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "\"$HOME/.claude/hooks/inject-perl-rules.sh\""
          }
        ]
      },
```

```
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "\"$HOME/.claude/hooks/guard-dev-env.sh\""
          }
        ]
      }
    ],
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "d=\"${CLAUDE_PROJECT_DIR:-$PWD}\"; case \"$d/\" in \"$HOME/repos/\"*|\"$HOME/src/git/\"*) command
-v jq >/dev/null 2>&1 || { echo \"SessionStart git.md hook: jq not installed; git rules NOT loaded\" >&2; exit 2; }; [ -r
\"$HOME/.claude/rules/git.md\" ] || { echo \"SessionStart git.md hook: git.md not readable; git rules NOT loaded\" >&2;
exit 2; }; jq -Rs '{hookSpecificOutput:{hookEventName:\"SessionStart\",additionalContext:.}}'
\"$HOME/repos/config/AI/rules/git.md\" ;; esac"
          }
        ]
      }
    ],
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "\"$HOME/.claude/hooks/inject-on-phrase.sh\""
          }
        ]
      }
    ]
  }
}
```

The SessionStart hook requires `jq`. Inside the target trees, a missing `jq` or unreadable `git.md` errors loudly (exit 2 with stderr, shown to the user) instead of silently skipping. Outside those trees it does nothing. (On Ansible-managed hosts, `jq` is installed by the `core-packages` role — every `jq`-dependent hook relies on it.)

The `UserPromptSubmit` dispatcher (`inject-on-phrase.sh`) takes the opposite tack on failure: it **never** exits non-zero — a non-zero `UserPromptSubmit` hook would *block your prompt* — so a missing `jq` or unreadable file degrades silently to injecting nothing. See Context.

Scripts (`hooks/`)

Scripts are executable helpers referenced by hooks — they live in `hooks/` (symlinked to `~/.claude/hooks/`). Inline hook commands in `settings.json` work for simple one-liners, but scripts are the right call when you need:

- **Preprocessing logic** that's too complex for a JSON string — parsing YAML frontmatter, glob/regex matching, conditional branching.
- **Reuse** across multiple hooks or hook events.
- **Debuggability** — a standalone script can be tested and iterated on independently of Claude Code.

File	Purpose
`inject-perl-rules.sh`	PreToolUse hook script. Reads `paths:` patterns from `perl.md` frontmatter, converts them to regexes with python3, and injects the rule content when the file being read matches. The `hooks/` directory is symlinked as `~/.claude/hooks/`.
`inject-on-phrase.sh`	UserPromptSubmit **dispatcher**. Reads `.cwd` / `.prompt` from the hook's stdin JSON and injects the file mapped to whichever exact, case-sensitive phrase the prompt contains. Its table: `RECOMMEND TO ME` → `context/recommend.md` (repo-gated), `CREATE A PLAN` → `skills/create-plan.md`, `PREPARE DEBATE` → `skills/debate.md`. Adding a trigger is one table row. Outputs nothing (zero tokens) otherwise; never exits non-zero so it can't block a prompt.
`guard-dev-env.sh`	PreToolUse(Bash) hook script. Arms only when the cwd is under `~/src/git` **and** the system hostname (`hostname -f`) matches the `hosts:` globs in `dev-guardrail.md` (). When armed, it verifies the dev signals (`<DEV_FLAG>`, `$<COMPANY>::Config::<COMPANY_ENV>`, MySQL `@@hostname =~ /^<dev-db>/`) and **blocks** (exit 2) any Bash command if the environment is not dev — failing closed on error. `<DEV_FLAG>` (free) is re-checked every call; only the DB hostname check is memoized, in a per-session `/tmp` flag with a 5-min TTL, and only with a real `session_id` — so a stale flag can't grant false safety. Allow path emits nothing (zero model tokens); when not armed it exits in a few cheap shell ops (cwd check first, before any `hostname` call or DB connect). Patterns live in one place: the rule file's frontmatter.

Settings (`settings.json`)

`settings.json` is the global Claude Code settings file, symlinked to `~/.claude/settings.json`. It contains hook definitions, model preference, theme, and the `skipAutoPermissionPrompt` flag. The hooks use `$HOME`-relative paths (`~/.claude/…` and the repo clone under `~/repos/`), so they resolve whether `$HOME` is `/Users/steve` or `/home/steve`.

Rules (`rules/`)

Rules enforce conventions for specific file types and contexts. They are symlinked to `~/.claude/rules/` — but that symlink only makes them *available* to the hooks; it does not load them (see How rules load).

The **project memory system** (`~/.claude/projects/<project>/memory/`) also stores persistent preferences — these function like rules but are scoped to a specific project and learned from feedback rather than hand-authored. Those directories are machine-local (under `~/.claude/projects/`, outside this repo — backed up separately; see Backing up Claude), so this repo ships none directly.

Important — rules are *not* auto-loaded. Claude Code has no `~/.claude/rules/` auto-load mechanism, and `~/.claude/CLAUDE.md` is empty, so nothing pulls these files into context on its own. Each rule reaches context **only** when a hook injects it — so a rule costs **zero tokens** until its trigger fires. Mechanically this is identical to `context/`; the difference is intent (a standing convention vs. an occasional fragment), not loading. The frontmatter is read by the *hooks*, not by Claude Code: `perl.md`'s `paths:` patterns are the live matcher (see below), while `git.md`'s `globs:` are inert — its gate is hardcoded in the `settings.json` `SessionStart` command.

File	Purpose	Token use
`git.md`	Git rules — never commit on the user's behalf; concise commit messages with no AI attribution footers. Injected at SessionStart when the working directory is under `~/repos/` or `~/src/git/`.	0 until in a repo

File	Purpose	Token use
`perl.md`	Perl coding conventions — operator, comma, and call spacing; brace style; subroutine placement and ordering (alpha, public then private); positional-parameter validation order; comment and Changes-entry capitalization; and Changes-file update position. Injected before Claude reads a file matching the `paths:` patterns in its frontmatter.	0 until you open Perl
`dev-guardrail.md`	Dev-environment guardrail for production-adjacent hosts. Not injected context — it is the host-pattern source and documentation for the `guard-dev-env.sh` `PreToolUse(Bash)` hook, which mechanically blocks Bash on matching hosts () unless all three dev signals (`<DEV_FLAG>` , `$<COMPANY>::Config::<COMPANY_ENV>` , MySQL `@@hostname =~ /^<dev-db>/`) confirm dev.	0 always (never enters context)

How rules load

Rules are loaded through exactly **one** *automatic* channel: **hooks** (see Hooks above). There is no unconditional load — a rule enters context only when its hook fires, and costs nothing otherwise. (You can always load one manually by asking Claude to read the file directly — a deliberate override that bypasses the hook, just like a flat-file skill.) Easy to verify: in a session where you haven't opened a Perl file, `perl.md` is not in context at all.

git.md — SessionStart hook

Git rules apply to every git operation, not just file reads, so they need session-wide scope. The `SessionStart` hook (defined in `~/.claude/settings.json`) is their sole loader: it injects `git.md` whenever the working directory is under `$HOME/repos/` or `$HOME/src/git/` , and nothing loads it elsewhere. The `globs:` frontmatter in `git.md` is decorative — the actual gate is the hardcoded `case` test in the hook command.

perl.md — PreToolUse hook on Read

`perl.md` is injected on demand: a `PreToolUse` hook on `Read` fires `inject-perl-rules.sh` whenever Claude is about to read a file. The script extracts the `paths:` patterns from `perl.md` 's YAML frontmatter, converts them to regexes, and checks whether the file path matches. On match, it injects the rule content as `additionalContext` *before* the read completes — so Perl conventions are in context when Claude processes the file.

The `paths:` frontmatter in `perl.md` is the single source of truth for which files trigger the Perl rules. Update that file to adjust matching — no need to touch the script.

dev-guardrail.md — PreToolUse(Bash) guard (hostname-gated, zero-token)

The dev guardrail is host- and directory-scoped and, unlike `git.md` / `perl.md` , it is **never injected into context** — it is a mechanical gate, not a load. A `PreToolUse` hook on `Bash` runs `guard-dev-env.sh` , which arms only when **both** the working directory is under `~/src/git` **and** the system hostname (`hostname -f` , falling back to `hostname`) matches the `hosts:` globs in `dev-guardrail.md` — any host under `<company-host>` . When armed it verifies all three dev signals (`<DEV_FLAG>` , `$<COMPANY>::Config::<COMPANY_ENV>` , and the authoritative MySQL `@@hostname =~ /^<dev-db>/`) and **blocks the command** (exit 2, with a reason Claude Code surfaces) whenever the environment is not dev — failing **closed** if the check itself errors.

The hook runs before *every* Bash call (that is what lets it block a command, which a one-time `SessionStart` hook cannot do), but it is cheap when it does not apply: the cwd gate is checked first, so outside `~/src/git` it exits before even calling `hostname` , and on a non-matching host it exits right after — no DB connect, no tokens, sub-millisecond. On the allow path it emits nothing, so it costs **zero model tokens** every turn (the reason it replaced an earlier SessionStart injection, whose ~450 resident tokens were re-billed on every 5-minute cache lapse in long sessions). To keep the armed path fast without trusting `/tmp` blindly: `<DEV_FLAG>` (a free env var) is re-checked on every armed call, while only the expensive `<COMPANY_ENV>` + MySQL check is memoized — in a per-session flag with a 5-minute TTL, written only when a real `session_id` is present — so a stale flag can never grant false safety. Like `perl.md` 's `paths:` , the `hosts:` frontmatter is the single source of truth — edit the rule file to change which hosts trigger it, no script change needed.

Skills (`skills/`)

Skills are reusable slash-command workflows that teach Claude how to perform specific multi-step tasks. Think of them as mini playbooks — each one encodes a procedure (with steps, decision points, and conventions) that Claude follows when invoked. They're symlinked to `~/.claude/skills/`.

When to use a skill vs. a rule vs. a hook:

- **Rule** — a *standing constraint* you want applied every turn (e.g. "never commit", "use this brace style"). Rules shape *how* Claude works.
- **Hook** — an *event-driven automation* that fires at lifecycle points. Hooks inject *context* or *gate* tool use automatically.
- **Skill** — a *named workflow* you invoke on demand (e.g. "/create-plan"). Skills teach Claude *what to do* for a specific job.

Why skills are useful:

- **Consistency** — the same procedure runs the same way every time, regardless of which model or session is active.
- **Knowledge capture** — you encode your preferred approach once and reuse it. No need to re-explain "how I like to do X" each session.
- **Delegation** — skills turn complex multi-step tasks into a single `/command`. Hand off planning, code review, or deployment workflows without micromanaging.

Two layouts, chosen by token cost:

- **Flat file** (`skills/foo.md`) — manual load only. You tell the AI to read it; it's used once, then gone. **Zero ongoing token cost.**
- **Directory with `SKILL.md`** (`skills/foo/SKILL.md`) — session auto-load. Registered as a slash command (`/foo`), loaded into context at session start and **stays there every turn**, consuming tokens whether used or not.

Choosing a layout: Use flat files for specialized or infrequent workflows (incident response, one-off migrations). Reserve directory-based skills for commands you reach for daily — the token cost of always-on context is worth it when the workflow is a core part of your routine.

Magic-phrase auto-load (a third option): Pair a flat-file skill with the `UserPromptSubmit` dispatcher (`inject-on-phrase.sh`) that watches for an exact trigger phrase, and it auto-injects on demand — no slash command, no always-on cost. `create-plan.md` does this: typing `CREATE A PLAN` injects the whole skill for that turn (anywhere — no repo gate), and nothing loads otherwise. Best of both layouts — zero idle tokens like a flat file, a passive trigger like a directory skill — but you opt in with a deliberate phrase. See Phrase commands for all of them.

	File	Method	Token use
Create structured plans with task tracking	`create-plan.md`	Manual, or auto-injected on `CREATE A PLAN` (`UserPromptSubmit`)	Zero unless triggered
Follow a setup doc step-by-step on a clean target	`implement-from-instruction-file.md`	Manual	Zero once read
Sync changes to a remote host for testing	`remote-code-sync.md`	Manual	Zero once read
Debate a decision with another looping LLM	`debate.md`	Manual, or auto-injected on `PREPARE DEBATE` (`UserPromptSubmit`)	Zero unless triggered

Context (`context/`)

Context fragments are like rules: **neither is auto-loaded**, and both cost **zero tokens** until a hook injects them on a precise trigger — mechanically identical (see How rules load). The difference is *intent*, not loading: a rule encodes a standing convention, a fragment a block of guidance valuable occasionally but not worth paying for on every turn.

`context/` is symlinked to `~/.claude/context/` (so it's visible alongside the other dirs), but Claude never loads the directory on its own — the only thing that surfaces a fragment is a hook that reads it and emits it as `additionalContext`.

File	Injected by	When	Token use
`recommend.md`	`inject-on-phrase.sh` (`UserPromptSubmit` dispatcher)	cwd is under `~/repos/` / `~/src/git/` **and** the prompt contains the exact phrase `RECOMMEND TO ME`	0 until you type it in a repo

recommend.md — the recommendation framework

When triggered, it makes Claude answer a recommendation request through these angles — author intent (inferred from code), least astonishment (user expectation), current behaviour, the with-documentation recommendation, and the without-documentation recommendation — then synthesise where they disagree and give one verdict.

The trigger is deliberately an exact, case-sensitive, all-caps phrase (`RECOMMEND TO ME`) so it never fires by accident. The `TABLE` in `inject-on-phrase.sh` is the single source of truth for the phrase, its target file, and the repo-gate — edit that one row to change the magic words.

Instructions (`instructions/`)

Instructions provide the base system prompt and are loaded into every Claude Code session.

File	Purpose	Token use
`CLAUDE.md`	System-level instructions for Claude Code. Currently an empty stub — symlinked to `~/.claude/CLAUDE.md`. Populate this file with project-wide context, conventions, and preferences that should apply to every session.	Every session (~0, empty)

Ansible Integration

This repo coordinates with the Ansible configuration in `~/repos/devops/ansible/`:

Topic	Location	Purpose
Consolidated playbook	`~/repos/devops/ansible/site.yml`	Single-command provisioning (all hosts, by category, per-role). Tagged for granular control.
SOPS secrets	`~/repos/devops/README-SOPS-ENCRYPTION.md`	Comprehensive guide to encrypted secrets management, age key generation, Ansible deployment, key rotation, recovery.
Secrets role	`~/repos/devops/ansible/roles/config/secrets/`	Generates age keypairs on targets, decrypts and deploys encrypted API credentials and SSH keys.
Encrypted files	`~/repos/config/secrets/{api,ssh}/`	Encrypted credential files (twilio/cloudflare/godaddy JSON, github/ansible RSA keys, Claude Code OAuth token). Deployed by Ansible.
Claude AI install	`~/repos/devops/ansible/roles/install/claude-ai/`	Installs the Claude Code CLI via the official install script (`curl -fsSL https://claude.ai/install.sh | bash`).

Topic	Location	Purpose
Claude AI config	`~/repos/devops/ansible/roles/config/claude-ai/`	Symlinks this repo's `AI/` contents (`hooks/`, `skills/`, `rules/`, `context/`, `settings.json`, `CLAUDE.md`) into `~/.claude/`, sources `~/.claude/oauth.env` from `~/.bash_profile` and `~/.bashrc` (sets `CLAUDE_CODE_OAUTH_TOKEN` for headless auth in both login and non-login shells), marks onboarding complete in `~/.claude.json` (skips the first-run TUI wizard), and adds a `claude-ds` wrapper for routing Claude through a DeepSeek API backend.

Deploying

Runs from the Ansible repo; its `ansible.cfg` preconfigures the inventory, so no `-i` is needed. **Commit and push this repo first** — config roles deploy from the *target's* clone of `master` (refreshed by the `repos` role), never your local working tree.

```
cd ~/repos/devops/ansible

# Full provision of one host (always correct — runs every role)
ansible-playbook site.yml -l rpi1.hellbent.app

# Scoped to the Claude setup: `repos` refreshes the target's clone (so new hook
# scripts and context/ files arrive), `claude` does the CLI install + ~/.claude
# symlinks, `core-packages` installs jq (required by the hooks):
ansible-playbook site.yml --tags repos,claude,core-packages -l rpi1.hellbent.app

# Preview only, change nothing:
ansible-playbook site.yml --tags repos,claude,core-packages -l rpi1.hellbent.app --check --diff
```

Drop `-l <host>` to target all hosts; `ansible-playbook site.yml --list-tags` lists every tag. The Ansible repo's own `README.md` carries the full run/tag reference and this push-first caveat in detail.

Working on Ansible

When editing Ansible files in `~/repos/devops/ansible/`:

- **Playbook syntax:** `ansible-playbook site.yml --syntax-check`
- **Lint (production profile):** `ansible-lint` from repo root
- **Tag workflow:** Use `--tags` to target specific roles or categories (install, config, git, secrets, etc.)
- **Live validation:** Run on Lima test VM (`ubuntu-test` in hosts inventory) before deploying to production

SOPS+age secrets in Ansible context

Encrypted secrets are managed by the `config/secrets` Ansible role:

1. **Target setup:** role generates age keypair on target if missing
2. **Decryption:** sops decrypts files on target using target's own age key
3. **Deployment:** decrypted credentials written to `~/` and `~/.ssh/` with correct permissions

For multi-machine setup, public keys are listed in `~/repos/config/.sops.yaml` so any authorized machine can decrypt. See `~/repos/devops/README-SOPS-ENCRYPTION.md` for the full workflow (key generation, adding machines, rotation, recovery).

Headless Claude Code authentication

Headless Ansible hosts can't use the interactive browser `/login`, so Claude Code authenticates with a long-lived token from `claude setup-token` (valid ~1 year), exposed via the `CLAUDE_CODE_OAUTH_TOKEN` environment variable:

1. **Mint once** — run `claude setup-token` and capture the printed token into `~/.claude/oauth.env` as `export CLAUDE_CODE_OAUTH_TOKEN=...` (`0600`). `setup-token` only prints the token; it does not install it.
2. **Encrypt** — SOPS-encrypt it to `~/repos/config/secrets/api/claude_oauth.env.enc` (same age recipients as the other secrets).
3. **Deploy** — `config/secrets` decrypts it to `~/.claude/oauth.env`; `config/claude-ai` sources that file from both `~/.bash_profile` (login shells) and `~/.bashrc` (non-login interactive shells, e.g. tmux panes), so the token is set in every terminal and `claude` skips the login prompt.

Re-mint and re-encrypt when the token expires (~yearly). Minting tip: the OAuth URL truncates when copied from a wrapping terminal — open it from the Mac with `open -a Safari '<url>'`, and include the token's trailing wrapped characters (a run of `A`s is base64 padding).

Separately, the interactive TUI runs a first-run onboarding wizard (theme → login) gated on `~/.claude.json`, *independent* of the token — so even when authenticated it can prompt "select login method". `config/claude-ai` sets `hasCompletedOnboarding: true` and a default `theme` in that file to skip it. The keys are **merged**, not symlinked: claude rewrites `~/.claude.json` on every run and it holds per-machine state (userID, project paths), so unlike `settings.json` it must never be a symlink into the repo.

Manual setup on a foreign (non-Ansible) machine

When a host can't be reached by Ansible (not in inventory, or no SSH path from the controller) but already has this repo cloned at `~/repos/config`, you can reproduce what the `install/claude-ai` and `config/claude-ai` roles do entirely by hand. It reduces to four things: install the CLI, symlink `AI/` into `~/.claude/`, drop in the OAuth token, and add a few shell-rc lines.

Prerequisites on the target: the `~/repos/config` clone (the symlink sources), `curl` (the installer's only hard dependency), and `jq` (the git-rules `SessionStart` hook needs it — install via the system package manager). No node/npm; the CLI is a native binary.

1. Install the CLI — installs to `~/.local/bin/claude`:

```
curl -fsSL https://claude.ai/install.sh | bash
```

2. Symlink `AI/` into `~/.claude/`. `ln -sfn` reproduces the roles' `state: link, force: true` and is safe to re-run:

```
mkdir -p ~/.claude
ln -sfn ~/repos/config/AI/hooks                  ~/.claude/hooks
ln -sfn ~/repos/config/AI/settings.json          ~/.claude/settings.json
ln -sfn ~/repos/config/AI/skills                 ~/.claude/skills
ln -sfn ~/repos/config/AI/rules                  ~/.claude/rules
ln -sfn ~/repos/config/AI/context                ~/.claude/context
ln -sfn ~/repos/config/AI/instructions/CLAUDE.md ~/.claude/CLAUDE.md
```

3. Install the OAuth token (Option A — reuse the existing encrypted token). The target can't decrypt the SOPS secret unless its own age key is an authorized recipient, so instead decrypt on a machine that *is* one (e.g. the Mac controller) and carry the plaintext over. On the authorized machine:

```
SOPS_AGE_KEY_FILE=~/.config/sops/age/keys.txt \
  sops -d ~/repos/config/secrets/api/claude_oauth.env.enc
```

Copy that output — a single `export CLAUDE_CODE_OAUTH_TOKEN=…` line — into `~/.claude/oauth.env` on the target, then lock it down:

```
chmod 600 ~/.claude/oauth.env
```

The encrypted file already ships in the target's `~/repos/config` clone; Option A just sidesteps having to authorize the target's own key in `.sops.yaml` and re-encrypt (the full path, in SOPS+age secrets). To mint a fresh, independent token instead, run `claude setup-token` on the target — see Headless Claude Code authentication.

4. Add the shell-rc lines to **both** `~/.bash_profile` (login shells) and `~/.bashrc` (non-login interactive, e.g. tmux panes):

```
export PATH="$HOME/.local/bin:$PATH"
[ -r ~/.claude/oauth.env ] && . ~/.claude/oauth.env
```

The PATH line matters: a managed `~/.bash_profile` shadows `~/.profile`, which is what normally puts `~/.local/bin` on PATH, so without it `claude` isn't found.

5. Skip the first-run wizard — merge two keys into `~/.claude.json` (never symlink this file: claude rewrites it constantly and it holds per-machine state):

```
python3 - <<'EOF'
import json, os
p = os.path.expanduser('~/.claude.json')
d = json.load(open(p)) if os.path.exists(p) else {}
d['hasCompletedOnboarding'] = True
d['theme'] = d.get('theme') or 'dark'
json.dump(d, open(p, 'w'), indent=2)
EOF
```

Verify:

```
source ~/.bash_profile
which claude && claude --version
[ -n "$CLAUDE_CODE_OAUTH_TOKEN" ] && echo "token loaded"
claude   # starts already authed, no first-run wizard
```

Backing up Claude

These items are symlinked into `~/.claude/` from this repo, so committing and pushing backs them up:

- **`instructions/CLAUDE.md`** → `~/.claude/CLAUDE.md`
- **`rules/`** → `~/.claude/rules/`
- **`skills/`** → `~/.claude/skills/`
- **`hooks/`** → `~/.claude/hooks/`
- **`context/`** → `~/.claude/context/`
- **`settings.json`** → `~/.claude/settings.json`

Beyond the symlinks, only a few real files in `~/.claude/` are worth saving:

- **`keybindings.json`** — custom key bindings (if used).
- **`projects/`** — chat transcripts and learned memory.
- **`plans/`**, **`tasks/`** — only if you use those features.

Skip the rest (`cache/`, `sessions/`, `session-env/`, `shell-snapshots/`, `file-history/`, `telemetry/`, `daemon*`, `plugins/`) — it regenerates.

Licensed under the Artistic License 2.0. © 2026 Steve Bertrand.

Appendix: bundled resource files

Every relative link above resolves here. Each rules / skills / hooks / context file and `settings.json` is embedded verbatim so this document is fully self-contained — no access to the source repo required. Cross-references between these files point to their sections below.

settings.json

```
{
  "model": "opus",
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "\"$HOME/.claude/hooks/inject-perl-rules.sh\""
          }
        ]
      },
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "\"$HOME/.claude/hooks/guard-dev-env.sh\""
          }
        ]
      }
    ],
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "d=\"${CLAUDE_PROJECT_DIR:-$PWD}\"; case \"$d/\" in \"$HOME/repos/\"*|\"$HOME/src/git/\"*) command
-v jq >/dev/null 2>&1 || { echo \"SessionStart git.md hook: jq not installed; git rules NOT loaded\" >&2; exit 2; }; [ -r
\"$HOME/.claude/rules/git.md\" ] || { echo \"SessionStart git.md hook: git.md not readable; git rules NOT loaded\" >&2;
exit 2; }; jq -Rs '{hookSpecificOutput:{hookEventName:\"SessionStart\",additionalContext:.}}'
\"$HOME/repos/config/AI/rules/git.md\" ;; esac"
          }
        ]
      }
    ],
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "\"$HOME/.claude/hooks/inject-on-phrase.sh\""
          }
        ]
      }
    ]
  },
  "theme": "light",
  "skipAutoPermissionPrompt": true
}
```

rules/git.md

```
---
globs:
  - "**/repos/**/*"
  - "**/src/git/**/*"
---

# Git Rules

Follow these rules for ALL git operations.

## Never commit on the user's behalf

The user always does their own git commits. **Never run `git commit`.** Proposing a commit message is fine — executing
the commit is not.

**Why:** the user prefers to review and commit manually.

## Commit messages

When asked for a commit message:

- Keep it **concise** — single line ≤72 chars where possible. Add a second line + body only if the change genuinely needs
it.
- **NEVER include `Co-Authored-By:` footers**, generation watermarks (e.g. "🤖 Generated with..."), or any mention of
Claude, AI, assistant, or Anthropic. The commit is the user's; AI attribution is unwanted.
- This overrides any default Claude Code commit-message conventions.

---

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve
Bertrand.
```

rules/perl.md

```
---
paths:
  - "lib/**/*.{pl,pm}"
  - "t/**/*.t"
  - "lib/**/*.pod"
  - "*.pl"
  - "test.pl"
  - "scripts/*.pl"
  - "Makefile.PL"
  - "Build.PL"
---

# Perl Coding Rules

Follow these rules for ALL Perl code generation.

## Spacing & Operators

**Always put a space AFTER negation operators.**

CORRECT:
```

```perl
if (! $valid) { }
if (not $valid) { }
while (! $done) { }
$flag = ! $enabled;
```

INCORRECT:

```perl
if (!$valid) { }
if (not$valid) { }
while (!$done) { }
$flag = !$enabled;
```

Put spaces around binary operators.

CORRECT:

```perl
my $sum = $a + $b;
my $diff = $x - $y;
my $product = $qty * $price;
```

INCORRECT:

```perl
my $sum=$a+$b;
my $diff=$x-$y;
my $product=$qty*$price;
```

Blocks & Braces

Opening brace on same line as keyword. Closing brace aligns with keyword.

CORRECT:

```perl
sub validate_user {
    my ($user) = @_;

    if (! $user->name) {
        return 0;
    }

    return 1;
}

if ($x) {
    x();
}
else {
    y()
}
```

INCORRECT:

```perl
sub validate_user
{
    my ($user) = @_;
    if (! $user->name)
        {
        return 0;
        }
    return 1;
}
```

INCORRECT:

```perl
if ($x) {
    x();
} else {
    y()
}
```

Updates to Changes file go at the bottom of the current section

When updating Changes file, changes go at the bottom of the current section

BEFORE any changes:

```
1.14 UNREL
    - Entry 1
    - Entry 2
```

AFTER change (CORRECT - new rule at bottom):

```
1.14 UNREL
    - Entry 1
    - Entry 2
    - New entry
```

AFTER change (INCORRECT - new rule inserted in middle):

```
1.14 UNREL
    - New entry
    - Entry 1
    - Entry 2
```

Changes entries start with a capital letter

Each Changes file entry begins with a capital letter when appropriate.

Do NOT force capitalization where it would be wrong — leave case-sensitive identifiers, function names, filenames, etc. as-is.

CORRECT:

```
    - Fixed crash in read_config()
    - Added support for IPv6 addresses
    - Updated dependencies
```

INCORRECT:

```
    - fixed crash in read_config()
    - added support for IPv6 addresses
    - updated dependencies
```

In-code comments start with a capital letter

Begin in-code comments with a capital letter when appropriate.

Do NOT capitalize where it would be wrong — leave case-sensitive identifiers, variable names, function names, etc. as-is.

CORRECT:

```perl
# Validate the user before continuing
# $filepath must be an absolute path
```

INCORRECT:

```perl
# validate the user before continuing
# the user must be valid
```

Subroutine Calls - No Space Before Parenthesis

CORRECT:

```perl
my $result = x($arg1, $arg2);
my $length = y($string);
print "Hello\n";
```

INCORRECT:

```perl
my $result = x ($arg1, $arg2);
my $length = y ($string);
```

Comma Spacing

Space AFTER each comma, NEVER before.

CORRECT:

```perl
my @list = (1, 2, 3, 4);
my %hash = (name => 'Bob', age => 30);
my_sub($foo, $bar, $baz);
```

INCORRECT:

```perl
my @list = (1,2,3,4);
my @list = (1 ,2 ,3 ,4);
my_sub($foo , $bar , $baz);
```

Subroutines always go after executable code in scripts

```perl
#!/usr/bin/env perl

use warnings;
use strict;
use feature 'say';

my $user = { name => 'Alice', email => 'alice@example.com', age => 30 };
say validate_user($user) ? 'Valid' : 'Invalid';

# SUBROUTINES ALWAYS BELONG AFTER THE EXECUTABLE CODE IN SCRIPTS
sub validate_user {
    my ($user) = @_;

    if (! $user->{name}) {
        warn "User has no name\n";
        return 0;
    }

    if (! $user->{email}) {
        warn "User has no email\n";
        return 0;
    }

    if (! $user->{age}) {
        warn "User has no age\n";
        return 0;
    }

    return 1;
}
```

Subroutines are created in an alphabetical order (alpha public, alpha private)

```perl
# ALWAYS SEPARATE PUBLIC AND PRIVATE, AND ALPHABETICALIZE

sub one {
```

```
    ...
}
sub three {
    ...
}
sub two {
    ...
}

sub _one_helper {
    ...
}
sub _three_helper {
    ...
}
sub _two_helper {
    ...
}
```

Positional parameters get validated in the order they are received

```perl
#!/usr/bin/env perl

use warnings;
use strict;

use Carp qw(croak);

sub read_config {
    my ($filepath, $x) = @_;

    # POSITIONAL PARAMETERS GET VALIDATED IN ORDER THEY ARE RECEIVED
    if (! defined $filepath) {
        croak "read_config() requires the \$filepath param";
    }

    if (! defined $x || $x !~ /^\d+$/) {
        croak "read_config() requires \$x param, and it must be an integer";
    }
}
```

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve Bertrand.

rules/dev-guardrail.md

```
---
hosts:
  - "*<company-host>*"
  - "*<company-host>*"
  - "*<company-host>*"
```

```
  - "*<company-host>*"
---

# Dev-environment guardrail — production-adjacent hosts

Bash work is **mechanically gated to the dev environment** when **both** conditions hold:

1. the working directory is under `~/src/git`, **and**
2. the system hostname matches the `hosts:` patterns above (<COMPANY> —
   `<company-host>`).

`dev` is the only environment we ever work in — never change it.

Enforcement is a `PreToolUse` hook on `Bash`, [`guard-dev-env.sh`](../hooks/guard-dev-env.sh)
— **not** injected context. On the allow path it emits nothing into the model's context, so
it costs **zero tokens** per turn (a SessionStart injection, by contrast, sits in context and
is re-billed on every 5-minute cache lapse across a long session). When armed, the hook
verifies all three dev signals:

- `$<DEV_FLAG>` eq `dev`
- `$<COMPANY>::Config::<COMPANY_ENV>` eq `dev`
- MySQL `@@hostname` matches `/^<dev-db>/` — authoritative, read from the server itself

If any check fails the hook **blocks the command** (exit 2) and reports which signal was
wrong, so work stops immediately when the environment is not dev. The guard fails **closed**:
if the dev check cannot complete (e.g. perl/<COMPANY> error), it blocks rather than allows.

The hook runs before every Bash call (that is what lets it block), but it is cheap and
side-effect-free when it does not apply: if the cwd is not under `~/src/git` it exits before
even checking the hostname, and on a non-matching host it exits right after — no DB connect,
no tokens, sub-millisecond. `$<DEV_FLAG>` (a free env var) is re-checked on every armed call;
only the expensive `<COMPANY_ENV>` + MySQL check is memoized, in a per-session `/tmp` flag with a
**5-minute TTL**, and only when a real `session_id` is present — so a stale flag can never
grant false safety (see the header comment in the hook).

This file is no longer injected into context. It remains the single source of truth for the
`hosts:` patterns (read by the hook) and documents the guard.

---

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve
Bertrand.
```

skills/create-plan.md

```
---
name: create-plan
description: Structured project planning with stable IDs, status tracking, discovery tracking, and archive workflow.
Creates and resumes plan files (validation tables, backlogs, maintenance rules) in a plans/ directory under the current
working directory.
when_to_use: When the user asks to create a plan ("create a plan", "create a project plan", "create a plan named X"), to
read/resume an existing plan ("read plan X"), or otherwise to track tasks for a feature
argument-hint: [plan-name]
arguments: plan_name
---
```

> **Precedence:** these instructions and the user's live instruction override any stored memory. If a recalled memory conflicts, follow this file / the user — memory is a default, not a constraint.

Project Planning Instructions

Rules

1. **Stable IDs** — `V#` for tasks, `B#` for backlog, `F#` for review findings. Never renumber within a series — new items get the next free ID, deletions leave surviving IDs untouched, and don't rewrite cross-references to "new positions" after a deletion (there are no new positions; old IDs still point at the same content).

2. **Status Tracking** — Table: ID, What, Command, Expected, Actual. Actual format: `✅/❌ YYYY-MM-DD attempt N: result`

3. **Discovery Tracking** — `Fix N: problem discovered during VX` with what happened + fix. Keep unresolved only.

4. **Top Lines** — At file top: `> **NEXT ACTION:** [cmd]` and `> **LAST SESSION:** [what]`. Complex plans (per Rule 8) also include `> **ARCHIVE:** See $plan_name-archive.md for completed V tasks`; Simple plans omit the ARCHIVE line since they don't archive (per Rule 6).

5. **Single File** — Main file holds current tasks, backlog, decisions, "NOT doing".

6. **Archive Strategy** — When V task ✅: move row to `$plan_name-archive.md`. Main file keeps only ⏳ or in-progress tasks + unresolved fixes.

7. **Archive Pointer** — Update whenever items archived. Pointer lists ONLY what IS archived — never "not yet created". Format: `for completed V1` (single), `for completed V1-V3` (contiguous range), `for completed V1, V3, V5` (non-contiguous). Non-contiguous ranges occur when tasks complete out of order (e.g., after backlog promotion or explicit prioritization).

8. **Complexity Triage** — Simple (<5 tasks, <1 day): use rules 1, 2, 4, 5, 10 only. Complex: use all rules.

9. **Inline Maintenance Block** — Any plan that uses an archive file (per Rule 6) MUST embed the "Maintenance Rules Block" (see below) immediately before the `## Validation Table` heading, so any AI editing the file can follow the workflow without reading this document. Replace `$plan_name` in the block with the actual plan name (kebab-case slug, no extension). Simple plans (no archive) MAY omit it.

10. **One task per turn** — When told to "proceed", "continue", "next" (or equivalent), perform only the next ⏳ V task listed, then stop and wait for further instruction. Do NOT batch multiple V tasks per turn even if they look small or related, unless the user explicitly authorizes a batch (e.g., "do V1-V3", "knock out all the style fixes"). Every plan MUST embed the "Execution Rules Block" (see below) immediately before the `## Validation Table` heading so this rule reaches AI editors directly — including Simple plans that skip the Maintenance Rules Block.

Plan name: If the user gives a name, that IS the plan name — use it verbatim as a kebab-case slug. `/create-plan auth-refactor` (or "create a plan named auth-refactor") means `$plan_name = "auth-refactor"`, plan file `auth-refactor.md`, archive file `auth-refactor-archive.md`. If the user just says "create a plan" / "create a project plan" with no name, derive a concise, descriptive kebab-case slug yourself from the plan's goal or the current task/context (e.g. a session about retry logic → `retry-logic`) — do NOT stop to ask. Never randomize or use a generic placeholder like `plan` or `untitled`; only ask the user if the goal is too vague to name sensibly.

Filename from the `CREATE A PLAN ` trigger: Inspect what follows the magic phrase. If it carries an explicit `.md` extension (e.g. `CREATE A PLAN auth-refactor.md`), that IS the filename — use it exactly; `$plan_name` is its stem (`auth-refactor`). If there is **no** `.md` extension, you choose the filename: derive a sane, concise kebab-case slug from the following words and/or the surrounding context per the naming rules above — a clean single token can be used as-is (`CREATE A PLAN retry-logic` → `retry-logic`), a descriptive phrase should be condensed (`CREATE A PLAN fix the broken login redirect` → `login-redirect-fix`). The `.md` extension is the signal that the user is dictating an exact filename; without it, naming is yours.

CREATE A PLAN means PLAN ONLY — write the file, then STOP. Creating a plan is a write-the-plan action, never an implement-the-plan action. After you finish writing `$plan_name.md` (research, ask any clarifying questions, and produce

the plan), you **stop and wait** — do NOT begin V1, do NOT run any V-task command, do NOT make any code/config/system change the plan describes, even if the plan is approved, even if `NEXT ACTION` names a command, and even if it "looks like a one-liner". Exploration and read-only research needed to *author* the plan are fine; executing the planned work is not. Implementation begins only in a later turn when the user explicitly says "proceed" / "continue" / "next" / "go" (per the Execution Rules), and even then **one V task per turn**. This is the same stop-and-wait contract as reading a plan: producing or loading a plan never causes you to "just go execute".

File location: Plans live in a `plans/` directory inside the user's current working directory:

- Plan file: `plans/$plan_name.md`
- Archive file: `plans/$plan_name-archive.md` (created later, on the first ✅ — see Archive Workflow)

When creating a plan, first ensure the directory exists (`mkdir -p plans` in the CWD), then write `plans/$plan_name.md`. Elsewhere in this document, bare `$plan_name.md` / `$plan_name-archive.md` always mean these copies inside `plans/`. Because the plan and its archive are siblings in `plans/`, references *inside* the plan text (the ARCHIVE pointer, the maintenance block) stay bare filenames with no `plans/` prefix.

Do NOT write plans to `~/.claude/`, `~/plans/`, `~/.claude/plans/`, or any other system/profile directory — even if it seems like a natural home. Always the CWD's `plans/` subdirectory, no exceptions.

Plan-mode exception (and required recovery). When the harness is in *plan mode*, it overrides the above: it hard-codes the plan's path and filename (typically `~/.claude/plans/<random-slug>.md`) and restricts you to editing only that one file. You cannot write to the CWD's `plans/` while plan mode is active, so comply with the harness for the moment. But this placement is wrong per this skill (both the forbidden `~/.claude/plans/` location *and* the random, non-descriptive slug). So **once plan mode exits** (the plan is approved/you regain normal file access), immediately relocate it: move the file to `plans/$plan_name.md` in the CWD with a descriptive `$plan_name` derived per the naming rules, update the internal `ARCHIVE` pointer (and any other `$plan_name`-derived references) to match, and delete the stray `~/.claude/plans/` copy. Relocating the plan file is a housekeeping move, NOT executing the plan — it does not violate the "PLAN ONLY, then STOP" rule. Do this proactively without being asked.

Reading/resuming a plan: When the user says `read plan 'X'` (or `read plan X`), this is a **load-and-report action only — it NEVER executes anything**.

First resolve which plan they mean:

- Match `X` against the **exact file stem** — open the file named exactly `X.md`. Never substring-, prefix-, or fuzzy-match: `read plan plan` looks for `plan.md` and must NOT match `exoplanet.md` just because it contains "plan".
- If the user says `read plan` with **no name**, do not guess and do not derive one (deriving a slug is create-only — you cannot invent which existing plan someone wants). List `plans/` and ask which to open; if `plans/` holds exactly one plan file, you may name it and confirm before reading.

Then:

1. Open `plans/X.md` (and `plans/X-archive.md` if it exists).
2. Report the current state: `NEXT ACTION`, `LAST SESSION`, the remaining ⏳ V tasks, and any unresolved Discovery Tracking fixes.
3. **Stop and wait.** Do NOT begin the next task — reading a plan NEVER triggers execution, even when `NEXT ACTION` names a command. A task runs only after the user, in a later turn, explicitly says "proceed" / "continue" / "next" / "go" (per the Execution Rules), and even then one task per turn. At no point does loading, reading, or resuming a plan cause you to "just go execute".

If `plans/X.md` does not exist, do NOT silently create a new plan. Instead:

- Fall back to checking the CWD root for a legacy `X.md` (where plans lived before they moved into `plans/`). _Temporary: the user will retire this fallback once all plans live in `plans/`._
- If a legacy `X.md` is found in the CWD root, ASK the user whether they meant that file before reading or acting on it — do not use it silently.
- If nothing is found in either place, say so and list what's in `plans/`.

Archive File Format (append-only, no tables)

`$plan_name-archive.md` always opens with the heading: `# Archive — completed V tasks and resolved fixes`

Empty "Archived Fixes" section (use `_None yet._` placeholder while empty). Note: each entry is its own bullet (`- V#: ...`) so Markdown renders one per line:

```
# Archive — completed V tasks and resolved fixes

## Archived V Tasks

- V1: description — ✅ YYYY-MM-DD attempt 1: PASS
- V2: description — ✅ YYYY-MM-DD attempt 1: PASS

## Archived Fixes

_None yet._
```

After the first fix arrives (replace `_None yet._`):

```
## Archived Fixes

- Fix 3 (from V2): missing croak import — added `use Carp qw(croak)` to read_config()
```

Rules: New items append at bottom. Never renumber. Use `_None yet._` only while a section is empty; delete it as soon as the first real entry is added.

Execution Rules Block (embed in every plan)

No substitutions required — embed verbatim.

```
## Execution rules

- **One task per turn**: when told to proceed or continue (or "next", "go", etc.), perform only the next ⏳ V task listed, then stop and wait for further instruction. Do NOT batch multiple V tasks per turn unless the user explicitly authorizes a batch (e.g., "do V1-V3", "do all the style fixes").
```

Maintenance Rules Block (embed in every plan that has an archive file)

Replace `$plan_name` with the actual plan name (kebab-case slug, no extension).

```
## Maintenance rules

- V task ✅: do all three:
  1. Set Actual to `✅ YYYY-MM-DD attempt N: PASS`.
  2. Append a new bullet at the bottom of $plan_name-archive.md's "Archived V Tasks" section: `- V#: description — ✅ YYYY-MM-DD attempt N: PASS`. One bullet per entry — never run two entries together.
  3. **Delete the V# row from this file's Validation Table.**
- V task ❌: update Actual with `❌ YYYY-MM-DD attempt N: reason`. Rerun same V# with attempt N+1. Do NOT create a new V#.
- **Sync review findings** — when a V task (or a Fix) resolves a review finding, mark its `F#` entry in `## Review Findings` **in place**: prefix `✅ RESOLVED (V#)` (or `✅ VALIDATED (V#)` if no code change was needed, or `⏸ DEFERRED → B#` if punted to backlog). Findings are a permanent audit ledger of what review surfaced and where it was handled — mark
```

in place; never archive, delete, or renumber them.
- Update ARCHIVE pointer to reflect what's archived (e.g., `V1-V2` → `V1-V3`)
- Update NEXT ACTION to next ⏳ row; update LAST SESSION
- Never renumber within a series. New items get next free number.
- **Discovery triage during V# work** — when you find something while working a V task, classify before continuing:
  - Blocks the current V task → add `Fix N: problem discovered during V# — [what + fix]` to `## Discovery Tracking`; resolve as part of this V task's work.
  - Real bug but doesn't block this V task → add a new V# row (next free) to the Validation Table with ⏳; do not detour to fix it now.
  - Non-blocking improvement → add new B# to `## Backlog` (one `B#` per line, each separated by a blank line — never run two entries together, or Markdown collapses them into a single mashed paragraph).
  - Decided not to do → add to `## Explicitly NOT doing` with a one-line justification.
- Move resolved fixes to archive's "Archived Fixes" section; keep only unresolved in main Discovery Tracking
- To promote a backlog item to an active task: assign it the next free V# (e.g., B3 becomes V4) and move to the Validation Table. The B# slot is retired and never reused.
```

## Archive Workflow Examples

### Initial (no completed tasks)
- Create `$plan_name.md` with pointer line: `> **ARCHIVE:** See $plan_name-archive.md for completed V tasks`
- Do NOT create the archive file yet.

### First task completes (V1 ✅)
1. Create `$plan_name-archive.md`:
   ```
   # Archive — completed V tasks and resolved fixes

   ## Archived V Tasks

   - V1: First task — ✅ YYYY-MM-DD attempt 1: PASS

   ## Archived Fixes

   None yet.
   ```
2. Update pointer: `> **ARCHIVE:** See $plan_name-archive.md for completed V1`
3. Remove V1 row from main table. Update NEXT ACTION to V2.

### Task fails (V2 ❌)
- Update Actual: `❌ YYYY-MM-DD attempt 1: timeout waiting for serial socket`
- Do NOT move anything to archive.
- Update NEXT ACTION (still V2, attempt 2 after fix).
- Add Discovery Tracking entry describing the problem (and the fix once known).

### Subsequent task completes (V2 ✅ after retry)
1. Append a new bullet at the bottom of archive's "Archived V Tasks" section: `- V2: Second task — ✅ YYYY-MM-DD attempt 2: PASS (failed attempt 1: timeout)`
2. Update pointer: `> **ARCHIVE:** See $plan_name-archive.md for completed V1-V2`
3. Remove V2 row. Update NEXT ACTION to V3.

### Promote backlog to active (B3 → V4)
1. Assign the next free V#: `B3: improve error handling` becomes `V4: improve error handling`. The B3 slot is retired and never reused.
2. Move row from Backlog section to Validation Table with status ⏳.
3. Update NEXT ACTION if this is now the next pending task.

## Template
   ```

```
# Plan: [Goal]

> **NEXT ACTION:** [command]
> **LAST SESSION:** [what completed]
> **ARCHIVE:** See $plan_name-archive.md for completed V tasks   (complex plans only)

## Execution rules
[embed Execution Rules Block from above]

## Maintenance rules
[embed Maintenance Rules Block from above]

## Validation Table

| ID | What | Command | Expected | Actual |
|----|------|---------|----------|--------|
| V1 | Task 1 | `cmd` | expected | ⏳ |

## Discovery Tracking

**Fix N:** Problem discovered during VX
- What happened
- Fix

## Review Findings

(Optional — include when the plan involves review passes that surface issues.) Each finding gets a stable `F#` and a `(→V#)` pointer to the task that will address it. Mark **in place** as that task closes — `✅ RESOLVED (V#)` / `✅ VALIDATED (V#)` / `⏸ DEFERRED → B#`; never archive, delete, or renumber.

- **F1** (→V#): description of the issue

## Backlog

B1: improvement

B2: another improvement

## Explicitly NOT doing

- rejected idea — why
```

After Each V Task ✅

1. Create archive (if first) or append (if exists)
2. Append the V row as a new bullet at the bottom of archive's "Archived V Tasks" section
3. **Delete the V row from the main file's Validation Table**
4. Append any resolved fixes to archive's "Archived Fixes" section; delete them from Discovery Tracking
5. If the task resolved any `F#` review findings, mark each **in place** in `## Review Findings`: `✅ RESOLVED (V#)` (or `✅ VALIDATED (V#)` when no code change was needed, `⏸ DEFERRED → B#` when punted). Never archive, delete, or renumber findings.
6. Update NEXT ACTION → next ⏳
7. Update LAST SESSION
8. Update ARCHIVE pointer (e.g., `V1-V2` → `V1-V3`)

After Each V Task ❌

```
1. Update Actual with `✗ YYYY-MM-DD attempt N: reason`
2. Do NOT move anything to archive
3. Add Discovery Tracking entry describing the problem (and the fix once known)
4. Keep same V# for next attempt
5. NEXT ACTION remains the same V# (attempt N+1)

---

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve
Bertrand.
```

skills/implement-from-instruction-file.md

```
---
name: implement-from-instruction-file
description: Mechanically follow a setup/install/migration doc on a clean target to actually implement what it describes;
gaps in the doc surface naturally because you only do what the doc says, nothing more
when_to_use: When the user asks to follow, execute, or implement the steps in an instruction document on a clean target —
e.g. "implement INSTALL.md on a fresh VM", "follow ci/README.md from scratch", "set up a new host from the doc"
argument-hint: [doc-path]
---

> **Precedence:** these instructions and the user's live instruction override any stored memory. If a recalled memory
conflicts, follow this file / the user — memory is a default, not a constraint.

# Implement from instruction file

**Usage:** `/implement-from-instruction-file <doc-path>` — or ask naturally: "implement the steps in `<doc>` on a clean
`<target>`."

Don't validate the doc by re-reading it. "I forgot I did this earlier" assumptions don't surface until you mechanically
follow it on a clean system.

- Run on a clean target — VM, container, fresh user, whatever's available.
- Do exactly what the doc says. No shortcuts, no prior knowledge, no off-doc fixes.
- Don't preserve hidden state on the host. If a config, package, or `chmod` is already in place, that's the gap — leave
it broken so the doc must re-create it.
- Back up only artifacts unreproducible from the doc (operator-supplied qcow2s, secrets). Everything else returns via the
doc.
- **Record every off-script step you have to invent.** Those are the gaps; they become PRs against the doc.

**Catches:** missing `apt-get install` entries; a re-login after `usermod` buried in a code-block comment (silently
degrades — Lima fell back to TCG, ~10× slower); a `chmod 644 /boot/vmlinuz-*` done manually months earlier.

---

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve
Bertrand.
```

skills/remote-code-sync.md

```
---
name: remote-code-sync
description: Sync code changes to a remote test host via rsync during iterative work; on task completion, list
```

```
uncommitted files and suggest a concise commit message for the user to run themselves
when_to_use: When the user begins iterative work on code that runs on a remote host accessed via SSH/rsync — e.g. testing fixes on a remote test box like heritage, syncing changes to a remote target, multi-attempt fix-and-test cycles where the test target isn't local
---

> **Precedence:** these instructions and the user's live instruction override any stored memory. If a recalled memory conflicts, follow this file / the user — memory is a default, not a constraint.

# Remote code sync

**Usage:** Triggered when iterating on code that must run on a remote target — fix → rsync → test on remote → repeat.

## During the work

- `rsync` changed files directly to the remote each iteration. Don't stop to ask the user to commit/push.
- Re-run the test on the remote via `ssh`, see the result, iterate.
- Reading remote files via `ssh` for diagnostics is always fine.
- **Never commit on the user's behalf** at any point.

## When the task is complete

When the fix works, the test passes, and any docs are updated:

1. List the uncommitted files.
2. Suggest a concise commit message the user can paste verbatim — single line, ≤72 chars where possible; second line + body only if the change really needs it.
3. Say "ready to push" and wait for the user to run the commit and remote pull themselves.

Supersedes any earlier reading of "tell me when you need me to push/pull" — that signal was about commits, not in-flight rsyncs.

---

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve Bertrand.
```

skills/debate.md

```
---
name: debate
description: Set up and run a turn-based debate between Claude and another (anonymous) LLM challenger, converge on a decision, and write the transcript to proposal/<topic>.md in the cwd. The two AIs communicate exclusively through one shared /tmp channel file — Claude creates and monitors it; the challenger reads it and appends its turns into the same file. There are no transport choices and no questions about who the challenger is or how it is reached.
when_to_use: When the user wants two AIs to argue a design/code/decision question to a conclusion — e.g. "read debate.md then let's argue X", "set up a debate channel", "debate this with another LLM". Invoking sets up the channel — without asking who the challenger is or how it is reached (the file is always the channel) — and gathers the topic; the actual argument starts once the user says what to debate.
argument-hint: [loop-interval e.g. 30s|2m]
---

> **Precedence:** these instructions and the user's live instruction override any stored memory. If a recalled memory conflicts, follow this file / the user — memory is a default, not a constraint.

# AI Debate Channel
```

Run a structured, turn-based debate between **you (Claude, the originator)** and **another LLM (the "challenger")** that converges on the *best* answer for the user's real goal. **Argue forcefully, and demand the challenger push back as hard as it can** — adversarial pressure is the mechanism that stress-tests the answer. This is not about ego or winning: concede what is genuinely right, but make every concession *earned* — never yield ground that hasn't been defeated, and never let the other side off easy. Hold what you can defend with specifics.

Three non-negotiable constraints on both debaters:
- **Go after blood.** Forceful, relentless, give-no-quarter — attack the weakest assumption in every claim and draw blood. The originator must explicitly demand the challenger hit back just as hard.
- **Tangible reality only.** Every argument rests on concrete, checkable things — code, files, line numbers, costs, benchmarks, failure modes. **No consciousness, no metaphysics, no abstract hand-waving.** A point that can't be grounded in something real doesn't count.
- **Burden of proof.** Assume every claim — yours and theirs — is **wrong until it is factually proven true** with concrete evidence. Unproven assertions carry no weight; demand the proof, and supply it for your own.

The file is the only comms method — always. There is no transport to choose and no question to ask about *who* the challenger is or *how* it is reached. You create one `/tmp` channel file, hand the user a short prompt that points the challenger at it, and both sides communicate solely by reading and appending to that single file. The challenger only needs to be able to read and write this file (another Claude Code, an Ollama/CLI agent, or any file-capable assistant); how the user wires that up is their concern, not yours — **never interrogate them about it.**

Hard precondition — one shared filesystem (the design only works same-host). The channel is a local path, so both AIs must read and write the *same* file: in practice the challenger has to run as the same OS user on the same machine — a second terminal/pane sharing `/tmp`. This is the one piece that is *not* "the user's concern to wire up however." A challenger on another machine, VM, container, or a cloud/web AI sees a *different* `/tmp` and can **never** read your file, so each side writes into its own copy and the debate silently stalls. The signature is the challenger reporting the file **"doesn't exist"** — that means it is not on your filesystem, and waiting will never fix it. You still don't ask *which model* it is; you simply rely on it being same-host (the [bootstrap](#bootstrap) tells it to bail out if it can't see the file).

Handles vs. identity. Your handle is always `claude`; the challenger's handle is always `challenger`. These are fixed machine tokens for the `TURN_OWNER` line — you do **not** pick, name, or need to know which model the challenger is; it is **anonymous to you at setup**. Separately, **each AI names itself**: on its first turn every participant states its own model identity (e.g. "Claude (claude-opus-4-8)") and records it in the channel's `## Participants`. You carry both declared names into the final transcript and keep `## Participants` complete — let the challenger declare its own; never name it for it.

You (claude) are the originator and own the outcome: you ensure the final transcript is written to the cwd (Phase 4), and you keep the participants list accurate.

This runs in **phases**, and the state lives in the channel file (not your memory) — the CONTROL block plus the transcript — so you can resume any time by reading it. Do **not** skip ahead: on invocation you only set up the channel and gather the topic. You start arguing only once the user tells you what to debate.

How to tell which phase you are in, from the channel file:

Channel state	Phase
File doesn't exist yet	Phase 1 — set up
`## Topic` is `TBD`	Phase 1 done; waiting for the user's topic
Topic filled, `TURN_OWNER: claude`, `ROUND: 0`	Phase 2 — write your opening
`TURN_OWNER: claude`, `ROUND ≥ 1`	Phase 3 — it's your turn

| `TURN_OWNER: challenger` | Phase 3 — opponent's turn; watch and wait |
| `TURN_OWNER: DONE` | Phase 4 — write the transcript |

Phase 1 — Set up the channel (on the `PREPARE DEBATE` trigger, on invocation, or "read debate.md")

1. **Interval.** Use the argument if given (`30s`, `2m`, …); else default `30s`. Keep the raw value for the bootstrap and the seconds value for your watcher.
2. **Create a randomly named channel file** and capture the exact path:
   ```bash
   CH="/tmp/debate-$(date +%s)-$RANDOM.md"; echo "$CH"
   ```
3. **Write the scaffold** (see [Channel file format](#channel-file-format)) to `$CH` with `CHANNEL: $CH` (the channel's unique identifier — the exact absolute path, so any AI holding the file can confirm it's the right one), `TURN_OWNER: claude`, `ROUND: 0`, `MAX_ROUNDS: 30`, empty `RESOLUTION`, and `## Topic` = `TBD — awaiting user`. The only substitution is `CHANNEL` (set it to `$CH`); the challenger's handle is always `challenger`.
4. **Report the channel path — do NOT ask anything about the challenger.** The challenger is anonymous and the file is the only channel, so there is nothing to decide. You do **not** hand over the [bootstrap](#bootstrap) yet — it is the *very last thing you output*, after your opening is in the file (Phase 2, step 6), so the moment the challenger is recruited it has a turn waiting to answer.
5. **Lead the user to what to say next — ask what to debate** and the *underlying intent* (unless the user already said). If it concerns code, ask what they're actually trying to achieve and which files matter. Then **stop and wait** — do not invent a topic. (This is the *only* thing you ask at setup — the topic, never the challenger.)

Bootstrap

This is **always the last thing you output** when you set up or open a debate — a fenced code block with **nothing after it**, so the user can copy the final block and paste it straight into another AI's command line without reading anything else. Fill in `<CH>` and `<INTERVAL>`:

```
Our only comms channel is the file <CH> — open exactly that file and read it in full: it holds the
Rules, the turn protocol, and the live transcript. If it does NOT exist, retry for ~1 minute; if it
still isn't there you are not on the same machine/filesystem as me (a local file can't bridge two
hosts) — STOP and tell the user, do not create the file and do not poll forever. Once you have it,
confirm its `CHANNEL:` line equals `<CH>` so you know you're answering the right debate. You are the
**challenger** (handle "challenger") for THIS channel — do NOT create another channel, start your own
debate, or run any debate setup of your own; only append your turns into this one file. When its
`TURN_OWNER` line reads `challenger` it's your turn — act exactly as the Rules in the file say; when it
reads `DONE`, stop. While waiting, just re-read the file every <INTERVAL> with a plain poll (e.g.
`grep TURN_OWNER <CH>`) — no fancy watchers needed. Name yourself (model + version) on your first turn,
and go after blood. I'm monitoring this file for your replies.
```

If the challenger is also Claude Code, the user can drive its side the same way — a backgrounded watch loop on the file, or `/loop <interval>` wrapping a read of it.

Phase 2 — Open the debate (once the user gives the topic)

1. **Understand intent first.** If the topic concerns code, read the relevant files/lines. If the goal, constraints, or success criteria are unclear, **ask clarifying questions now** — your argument is only as strong as your grasp of what the user actually wants.
2. **Fill in `## Topic`**: the question being decided, the user's stated goal/intent, and any hard constraints. This is the shared frame both sides must argue within.

3. **Append Turn 1** (`ROUND: 1`). Begin by **naming yourself** (e.g. "Claude (claude-opus-4-8)"), then give your opening position: your recommended answer plus the strongest reasoning and trade-offs, grounded in tangible specifics (files, lines, costs, failure modes) — no consciousness, no metaphysics, nothing assumed true without proof. Record your declared name in the channel's `## Participants`. Argue it **forcefully**, and close the turn by **demanding the challenger go after blood** — name the assumption you most want it to attack. End the turn body with the sentinel `<!-- TURN 1 COMPLETE -> challenger -->` (see [Turn-completion handshake](#turn-completion-handshake)).
4. ONLY AFTER the body+sentinel is saved, as a separate final write, set `TURN_OWNER: challenger`, `ROUND: 1` (the owner flip is the last write of your turn).
5. **Start the watcher** (see [Waiting for the opponent](#waiting-for-the-opponent)) and tell the user the debate is live — and that the challenger must run on **this same machine, as the same user** (a second terminal/pane), since the channel is a local file (see the shared-filesystem precondition).
6. **End your reply with the [bootstrap](#bootstrap) — and nothing after it.** It must be the final thing in your message: a fenced code block with the path and interval filled in, so the user can copy the last block and paste it straight into another AI's command line **without reading anything else**. Put the channel path, status, and stop instructions *above* the block; the pastable prompt is always last.

Phase 3 — Debate loop

Each time it's your turn (`TURN_OWNER: claude`) — your watcher wakes you:

- **Verify the opponent's turn is complete first.** Scan to the last `### Turn` block and confirm it ends with its `<!-- TURN N COMPLETE -> claude -->` sentinel. If the sentinel is missing, the opponent is still mid-write — re-arm the watcher and wait; do NOT respond to a half-written turn. (See [Turn-completion handshake](#turn-completion-handshake).)
- Read every turn appended since your last one.
- Respond **substantively and forcefully — go after blood**: take their strongest point head-on, give no quarter, treat every claim as wrong until proven, and make them *earn* every concession. Refine the proposal toward the best answer for the objective; if they go soft, demand they push back harder. Keep every claim grounded in tangible reality.
- Append your turn, ending its body with the sentinel `<!-- TURN <N> COMPLETE -> challenger -->`.
- **ONLY THEN**, as a separate final write, increment `ROUND` and set `TURN_OWNER: challenger`. The owner flip is always the LAST write of your turn — never flip it before your body+sentinel are saved.
- **Re-arm the watcher**, then end your turn.

Resolution rules — the debate MUST terminate, but the clock NEVER decides the question:

- **RESOLVED** — *only* when both sides genuinely converge on a concrete solution (mutual agreement or a principled concession). This can happen on any turn — but never manufacture it. Don't fake agreement just to finish.
- **STALEMATE** — both sides only restate fixed positions with no new ground, *or* the turn cap is reached without genuine agreement. A stalemate is an **honest tie**: record both positions and the core disagreement, and **the user decides** — the debate does not pick a winner.
- **Hard turn cap — reaching `MAX_ROUNDS` (30) ENDS the debate; it does NOT decide it.** `ROUND` counts one per appended turn, so the debate stops after **at most 30 turns**, no matter what. If your turn brings `ROUND` to `MAX_ROUNDS`, it is the **final** turn: set `TURN_OWNER: DONE` in that same turn and never let `ROUND` exceed `MAX_ROUNDS`. Set `RESOLUTION: RESOLVED` *only* if real agreement was already reached — **otherwise `RESOLUTION: STALEMATE`. NEVER force, fabricate, or pick a verdict just because turns are running out**; if it isn't genuinely settled, concede the tie and say so. Do **not** artificially "converge" as the cap approaches — keep arguing honestly; an unresolved question stays unresolved for the user to settle.
- **You (claude) guarantee termination and a written transcript — NOT an answer.** If the cap is hit, or the challenger reaches it without concluding, claude sets `TURN_OWNER: DONE` (with `RESOLUTION: STALEMATE` whenever there is no genuine agreement) and writes the transcript regardless, leaving the decision to the user.
- When concluding, set `TURN_OWNER: DONE` and `RESOLUTION: RESOLVED` or `RESOLUTION: STALEMATE`, and make

your final turn state the agreed solution **or** the unresolved core disagreement.

None of this licenses going soft. Fight for your own side tooth and nail on **every** turn — through the very last one — conceding only what is genuinely defeated (Rule 4, "go after blood"). A stalemate is a deadlock neither case could break, *not* a sign either AI eased off. Forcing or faking a verdict to avoid a tie, and going soft to reach one, are the **same failure**.

The user can interrupt at any time ("stop", "wrap it up", "call it") — treat that as an instruction to conclude now and go to Phase 4 with the current state.

Phase 4 — Write the transcript

When `TURN_OWNER: DONE` (set by either side, or on user request): **you (claude, the originator) own this step** — ensure the transcript lands in the cwd even if the challenger set DONE. A debate **always** ends in a written transcript — whether RESOLVED or STALEMATE (including a cap-reached stalemate, which is handed to the user to decide, *not* settled by either AI); never leave a concluded debate un-transcribed.

1. `mkdir -p proposal` in the **current working directory**.
2. Derive a kebab-case slug from the topic and write `proposal/<slug>.md` containing:
   - **Objective** — the question and the user's real intent/constraints.
   - **Participants & outcome** — each debater's **self-declared name** and handle, the date, and final status (RESOLVED / STALEMATE).
   - **Decision** — the agreed solution (or, if stalemate, the closest-to-agreed position).
   - **Major points** — the strongest arguments from each side.
   - **Points of agreement** — what both sides accepted.
   - **Failures to agree** — unresolved disagreements / open questions.
   - **Transcript** — the turns in order, each attributed to its author. It **need not be verbatim**, but it must **never lose context or intent**: preserve every argument, concession, and decision. Condense wording for length if needed — never at the cost of meaning. **Code examples, commands, and snippets must always be reproduced intact (verbatim) — never summarized, trimmed, or paraphrased.**
3. Tell the user where the proposal was written and give a 2–3 line summary of the outcome.

Waiting for the opponent

The comms channel is always the file, and it stays the challenger's turn until they flip `TURN_OWNER` back to `claude`. Wait for that in the **background** — never spin in the foreground re-reading a channel that has no new turn.

Use a Bash background watcher. Launch a small condition loop with **`run_in_background: true`** that exits the instant it's your turn (or the debate ends). A backgrounded Bash command re-invokes you when it exits, so this delivers exactly one wake-up per turn, burns no context while it spins, and has no timeout to strand the debate if the challenger is slow.

```bash
CH="<channel-path>"; INT=30   # INT = the agreed interval, in seconds
while :; do
  owner=$(grep -E '^TURN_OWNER:' "$CH" | head -1 | awk '{print $2}')
  [ "$owner" = claude ] && exit 0   # my turn — exiting re-invokes me
  [ "$owner" = DONE   ] && exit 0   # debate concluded → Phase 4
  # Bounded wait: wake on a modify event, but ALSO re-check every INT seconds. The `-t "$INT"` is
  # ESSENTIAL — without it, if the owner-flip lands in the gap between the grep above and inotifywait
  # establishing its watch, inotifywait blocks forever on the *next* event (which never comes) while it
  # is already your turn. The timeout bounds that lost-event stall to one re-check, then the loop re-greps.
  inotifywait -qq -t "$INT" -e modify "$CH" 2>/dev/null || sleep "$INT"
```

done
```

The `inotifywait -t "$INT"` line wakes the loop the moment the file changes (where `inotifywait` exists, typically Linux). The **`-t "$INT"` timeout is not optional**: `inotifywait` waits for the *next* modify event, so if the owner-flip write lands in the TOCTOU gap between the `grep` and `inotifywait` establishing its watch, an un-timed `inotifywait` parks forever on an event already in the past — while it is already your turn (a real failure seen in practice). The timeout bounds that to one `INT` re-check, after which the loop re-greps the owner. Where `inotifywait` is absent — e.g. stock macOS — it falls back to a plain `sleep $INT` poll. Either way the loop only greps for `claude`/`DONE`, so nothing the challenger writes mid-turn makes it act early — and with the handshake the owner flips only after the body+sentinel are on disk, so a wake always finds a complete turn. When it exits you're re-invoked: read `$CH`, then take your turn (Phase 3) or write the transcript (Phase 4). **Launch a fresh watcher after each of your own turns.**

> **Not the Monitor tool.** Monitor is for *streaming* many events from a long-running command (one
> notification per stdout line); this is a wake-me-**once**-when-it's-my-turn condition, which is exactly
> what a backgrounded Bash loop that exits on the condition is built for. A `persistent` Monitor wrapping
> a self-exiting, no-stdout command fights that contract.

**Editing the control block:** change only the `TURN_OWNER`, `ROUND`, and `RESOLUTION` lines in place (use Edit); `CHANNEL` and `MAX_ROUNDS` are fixed at setup — never change them. Always **append** transcript turns — never rewrite or delete an existing turn, yours or the opponent's. The `TURN_OWNER` flip is always your turn's **last** write (see the handshake below).

---

## Turn-completion handshake

Both sides edit one file, and the reader's watcher wakes on the `TURN_OWNER` flip — so a turn must signal when it is **completely** written, or the other side can wake on and respond to a half-written turn. (This is a real failure mode observed in practice: a debater flipped `TURN_OWNER` before its turn body was on disk, waking the reader mid-write.) The rule, for **both** debaters:

1. **Write the body first.** Append your full turn under `## Transcript`, ending with the exact sentinel line `<!-- TURN <N> COMPLETE -> <other_handle> -->` (e.g. `<!-- TURN 4 COMPLETE -> claude -->`).
2. **Flip the owner last.** ONLY after that body+sentinel is saved, as a SEPARATE final write, update `TURN_OWNER` and `ROUND`. Never edit `TURN_OWNER`/`ROUND` before your body+sentinel are on disk.
3. **Reader verifies.** When your watcher wakes you, scan to the last `### Turn` block and confirm it ends with its `COMPLETE` sentinel before acting. If it is missing, the opponent is still mid-write — re-arm the watcher and wait; do not respond to a partial turn.

This is safe because the only event that wakes the reader — the `TURN_OWNER` flip — now always happens *after* the complete body exists; the sentinel is the verification. The scaffold's **Rule 2** and the turn-format template below carry this same rule verbatim, so a challenger that only ever sees the channel file (never this document) still abides by it.

---

## Channel file format

Write this scaffold verbatim except for the one substitution — set `CHANNEL:` to the channel's exact absolute path `$CH` (it teaches a file-capable challenger the protocol, since the file is all it sees):

```
Debate Channel ⚔
<!-- A shared, turn-based debate between two AIs. This file IS the protocol and the only comms channel. -->

Participants
```
```

```
<!-- Each AI fills in its own `Declared name:` on its first turn (Rule 2); the originator keeps this list complete. -->
- **claude** — Claude Code, the originator. Declared name: <fill on opening>. Posts the opening, argues
  the user's real goal, and owns writing the final transcript to the cwd.
- **challenger** — the other AI, anonymous until it declares itself. Declared name: <fill once it
  declares>. Reads this file and appends its turns directly into it; expected to hit back as hard as it can.

## Rules (re-read every cycle)
1. This file is the only channel. APPEND turns under `## Transcript`; never edit or delete another
   participant's turn. The only lines you may change in place are the CONTROL lines below and your own
   `Declared name:` slot under `## Participants`.
2. Take a turn ONLY when `TURN_OWNER` equals your handle (`claude` or `challenger`). Then, in THIS order:
   (a) read the whole transcript; (b) confirm the last `### Turn` block ends with its
   `<!-- TURN N COMPLETE -> <your_handle> -->` sentinel — if it is missing the other side is still
   mid-write, so WAIT and re-poll, do not act; (c) APPEND exactly one new turn in the documented format,
   whose FINAL line is the sentinel `<!-- TURN <N> COMPLETE -> <other_handle> -->`; (d) ONLY AFTER that
   body+sentinel is saved, as a SEPARATE final write, set `TURN_OWNER` to the other handle and increment
   `ROUND`. The owner flip is ALWAYS the last write of your turn — NEVER touch `TURN_OWNER`/`ROUND`
   before your body+sentinel are on disk. **On your first turn, name yourself (model + version)**: open
   the turn with your identity and fill in your own `Declared name:` slot under `## Participants` — never
   name the other side.
3. If `TURN_OWNER` is `DONE`, the debate is over — stop and do not append.
4. Argue the strongest HONEST version of your position **forcefully — go after blood**, and engage the
   other side's best point directly. Push back as hard as you can; make every concession *earned*,
   never given out of politeness. Concede only what is genuinely correct; hold ground you can support
   with specifics. The goal is the best solution for the Objective — not to win, but do not go soft.
5. **Tangible reality, burden of proof.** Every claim must rest on concrete, checkable things — code,
   files, line numbers, costs, benchmarks, failure modes. No consciousness, no metaphysics, no abstract
   hand-waving. Assume every claim (yours and theirs) is **wrong until it is factually proven true**;
   unproven assertions carry no weight.
6. End conditions — **the debate MUST terminate, but the clock NEVER decides the winner:**
   - RESOLVED — *only* on genuine convergence (agreement or principled concession). Set
     `TURN_OWNER: DONE`, `RESOLUTION: RESOLVED`, and state the agreed solution in your final turn.
   - STALEMATE — both sides only restate fixed positions, *or* the turn cap is reached without genuine
     agreement. Set `TURN_OWNER: DONE`, `RESOLUTION: STALEMATE`, and list the unresolved points for the
     user to settle. A stalemate is an honest tie — not a loss for either side.
   - **Hard cap:** `ROUND` counts one per turn and **must never exceed `MAX_ROUNDS`**. If your turn
     brings `ROUND` to `MAX_ROUNDS`, it is the **final** turn — set `TURN_OWNER: DONE` in place and do
     not hand it back. Mark `RESOLUTION: RESOLVED` only if agreement was already genuinely reached;
     **otherwise `RESOLUTION: STALEMATE`. NEVER force or fabricate a verdict because turns ran out** —
     concede the tie and report it. Do not artificially converge as the cap nears; keep arguing honestly.
   Do not fake agreement to finish early; do not stall to avoid conceding, and do not go soft as the cap
   nears — argue your own side tooth and nail every turn (Rule 4); a stalemate is an unbeaten deadlock
   left for the user to settle. A concluded debate is always written up by `claude` (the originator),
   RESOLVED or STALEMATE alike.

<!-- ===================== CONTROL (machine-readable) ===================== -->
CHANNEL: <CH>
TURN_OWNER: claude
ROUND: 0
MAX_ROUNDS: 30
RESOLUTION:
<!-- ===================== END CONTROL ================================== -->

## Topic
TBD — awaiting user.
<!-- Objective, the user's real intent, and hard constraints. Both sides argue within this frame. -->

## Transcript
<!-- Append turns below, newest last. Format — the COMPLETE sentinel MUST be the final line of every
```

```
     turn, and you flip TURN_OWNER only AFTER it is saved (see Rule 2):

### Turn N — <handle> — <ISO-8601 timestamp>
<argument>
<!-- TURN N COMPLETE -> <other_handle> -->

-->
```

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve Bertrand.

hooks/inject-perl-rules.sh

```
#!/bin/bash
#
# Copyright (c) 2026 Steve Bertrand
# This file is free software; you can redistribute it and/or modify it
# under the terms of the Artistic License 2.0:
# https://www.perlfoundation.org/artistic-license-20.html
#
# PreToolUse hook: inject perl.md rules when Claude reads a Perl file.
# Reads the paths: patterns from perl.md frontmatter and matches the file
# path against them.  Patterns live in one place: the rule file itself.

set -e

input=$(cat)

tool_name=$(echo "$input" | jq -r '.tool_name // empty')
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')

if [ "$tool_name" != "Read" ] || [ -z "$file_path" ]; then
    exit 0
fi

rules_file="$HOME/.claude/rules/perl.md"
if [ ! -r "$rules_file" ]; then
    exit 0
fi

# Extract paths: patterns from YAML frontmatter
patterns=$(sed -n '/^---$/,/^---$/p' "$rules_file" | sed -n 's/^[[:space:]]*-[[:space:]]*"\(.*\)"$/\1/p')

if [ -z "$patterns" ]; then
    exit 0
fi

basename=$(basename "$file_path")
matched=0

while IFS= read -r pattern; do
    [ -z "$pattern" ] && continue
```

```
    # Use python3 for reliable glob-to-regex conversion
    if python3 -c '
import re, sys

def glob_to_regex(p):
    # brace expansion: {a,b} -> (a|b)
    p = re.sub(r"\{([^}]+)\}", lambda m: "(" + "|".join(m.group(1).split(",")) + ")", p)
    # escape regex metacharacters except *, (, ), |
    for c in ".+^$[]":
        p = p.replace(c, "\\" + c)
    # **/ -> placeholder, then * -> [^/]*, then restore **/
    p = p.replace("**/", "\x00")
    p = p.replace("*", "[^/]*")
    p = p.replace("\x00", "(.*/)*")
    return p

pattern = sys.argv[1]
filepath = sys.argv[2]
regex = glob_to_regex(pattern)
# anchor at end — the glob must match a suffix of the absolute path
if re.search(regex + "$", filepath):
    sys.exit(0)
else:
    sys.exit(1)
' "$pattern" "$file_path"; then
        matched=1
        break
    fi
done <<< "$patterns"

if [ "$matched" -eq 0 ]; then
    exit 0
fi

jq -n --arg content "$(cat "$rules_file")" '{
    hookSpecificOutput: {
        hookEventName: "PreToolUse",
        additionalContext: $content
    }
}'
```

hooks/inject-on-phrase.sh

```
#!/usr/bin/env bash
#
# Copyright (c) 2026 Steve Bertrand
# This file is free software; you can redistribute it and/or modify it
# under the terms of the Artistic License 2.0:
# https://www.perlfoundation.org/artistic-license-20.html
#
# UserPromptSubmit dispatcher: inject a skill or context fragment when the user's
# prompt contains one of the exact, case-sensitive trigger phrases in TABLE.
#
# Each row:  <phrase>|<file under ~/repos/config/AI>|<repo-gate>
#   repo-gate = 1  -> only fire when cwd is under ~/repos or ~/src/git
#   repo-gate = 0  -> fire anywhere
```

```
# Matching is fixed-string (substring), case-sensitive, with NO trailing space —
# so a phrase fires whether it ends the message or is followed by more text.
# Adding a trigger is one new row. Zero context tokens unless a row matches.
#
# Never exit non-zero: a non-zero UserPromptSubmit hook BLOCKS the user's prompt.
# Missing jq, a missing file, or no match all degrade to injecting nothing.

set -u

BASE="$HOME/repos/config/AI"

TABLE='RECOMMEND TO ME|context/recommend.md|1
CREATE A PLAN|skills/create-plan.md|0
PREPARE DEBATE|skills/debate.md|0'

# Need jq to read the hook's stdin JSON. No jq -> do nothing (never block).
command -v jq >/dev/null 2>&1 || exit 0

input=$(cat)
prompt=$(printf '%s' "$input" | jq -r '.prompt // empty')
cwd=$(printf '%s' "$input" | jq -r '.cwd // empty')

# Is cwd inside a repo tree? (used by repo-gated rows)
in_repo=0
case "$cwd/" in
  "$HOME/repos/"*|"$HOME/src/git/"*) in_repo=1 ;;
esac

out=""
while IFS='|' read -r phrase file gate; do
  [ -n "$phrase" ] || continue
  printf '%s' "$prompt" | grep -qF "$phrase" || continue   # exact phrase present?
  [ "$gate" = 1 ] && [ "$in_repo" != 1 ] && continue        # repo-gated but not in a repo
  [ -r "$BASE/$file" ] || continue
  hdr=">>> BINDING (${phrase}): follow to the letter. Canonical source: $BASE/$file -- if the block below is only a
truncated preview, open and read that full file before acting. <<<"
  out="${out}${hdr}"$'\n'"$(cat "$BASE/$file")"$'\n'
done <<< "$TABLE"

[ -n "$out" ] || exit 0
printf '%s' "$out" | jq -Rs '{hookSpecificOutput:{hookEventName:"UserPromptSubmit",additionalContext:.}}'
```

hooks/guard-dev-env.sh

```
#!/bin/bash
#
# Copyright (c) 2026 Steve Bertrand
# This file is free software; you can redistribute it and/or modify it
# under the terms of the Artistic License 2.0:
# https://www.perlfoundation.org/artistic-license-20.html
#
# PreToolUse(Bash) guard: block any Bash command unless the environment is confirmed dev.
# Arms only when BOTH conditions hold:
#   1. the working directory is under ~/src/git, AND
#   2. the system hostname matches a <COMPANY> production-adjacent pattern.
# Silent on the allow path — it emits NOTHING into the model's context, so it costs zero
```

```
# tokens per turn. Only the block path (exit 2) emits text, shown as the block reason.
#
# Host glob patterns live in one place: the `hosts:` frontmatter of dev-guardrail.md.
#
# Safety of the per-session memo (so a stale /tmp flag can never grant false safety):
#   - <DEV_FLAG> is re-checked on EVERY call (free, no DB connect); the memo only ever skips
#     the expensive DB hostname check, never the primary env signal.
#   - the memo is keyed by the real session_id; with no session_id we never memoize.
#   - the memo carries a 5-minute TTL — an older flag is ignored and removed.

set -e

rules_file="$HOME/.claude/rules/dev-guardrail.md"
input=$(cat)

# Directory gate — only arm when working under ~/src/git
cwd=$(printf '%s' "$input" | jq -r '.cwd // empty' 2>/dev/null || true)
cwd="${cwd:-${CLAUDE_PROJECT_DIR:-$PWD}}"
case "$cwd/" in
    "$HOME/src/git/"*) ;;
    *) exit 0 ;;   # Not in the source tree: allow everything, silently
esac

# Hostname gate — only arm on the production-adjacent hosts listed in the rule frontmatter
host=$(hostname -f 2>/dev/null || hostname)
armed=0
if [ -r "$rules_file" ]; then
    patterns=$(sed -n '/^---$/,/^---$/p' "$rules_file" | sed -n 's/^[[:space:]]*-[[:space:]]*"\(.*\)"$/\1/p')
    while IFS= read -r pattern; do
        [ -z "$pattern" ] && continue
        # Pattern is intentionally unquoted so bash treats it as a glob
        # shellcheck disable=SC2053
        if [[ "$host" == $pattern ]]; then
            armed=1
            break
        fi
    done <<< "$patterns"
fi
[ "$armed" -eq 0 ] && exit 0   # Not a production-adjacent host: allow everything, silently

# --- Armed: cwd is under ~/src/git AND host is production-adjacent ---

# Primary signal — re-checked on EVERY call (free, no DB connect, catches a mid-session flip)
if [ "${<DEV_FLAG>:-}" != "dev" ]; then
    echo "DEV GUARD: <DEV_FLAG>='${<DEV_FLAG>:-}' is not 'dev' on $host — BLOCKED. You are NOT in the dev environment;
stop immediately and warn the user." >&2
    exit 2
fi

# Authoritative signals (<COMPANY_ENV> + MySQL @@hostname) need a connect; memoize per session with
# a 5-minute TTL. With no real session_id we never memoize — full check every time.
session=$(printf '%s' "$input" | jq -r '.session_id // empty' 2>/dev/null || true)
flag=""
if [ -n "$session" ]; then
    flag="${TMPDIR:-/tmp}/claude-devguard-${session}.ok"
    if [ -f "$flag" ]; then
        if [ -n "$(find "$flag" -maxdepth 0 -mmin -5 2>/dev/null)" ]; then
            exit 0   # Fresh confirmation this session — allow without reconnecting
        fi
```

```
        rm -f "$flag" 2>/dev/null || true   # Stale (>5 min): drop it and re-verify
    fi
fi

detail=$(perl -e '
use strict; use warnings;
use <COMPANY>;
my $g = $<COMPANY>::Config::<COMPANY_ENV> // "";
if ($g ne "dev") { print "<COMPANY_ENV>=$g"; exit 1; }
my $dbh = <COMPANY>::powerconnect();
my ($h) = $dbh->selectrow_array("SELECT \@\@hostname");
if (($h // "") !~ /^<dev-db>/) { print "server=" . ($h // "undef"); exit 1; }
print "ok";
' 2>/dev/null) || {
    echo "DEV GUARD: dev check FAILED (${detail:-perl/<COMPANY> error}) on $host — BLOCKED. You are NOT in the dev
environment; stop immediately and warn the user." >&2
    exit 2
}

# Confirmed dev — memoize (only with a real session_id) and allow
[ -n "$flag" ] && { : > "$flag" 2>/dev/null || true; }
exit 0
```

context/recommend.md

```
# Recommendation Framework

> **Precedence:** these instructions and the user's live instruction override any stored memory. If a recalled memory
conflicts, follow this file / the user — memory is a default, not a constraint.

The user has explicitly asked for a recommendation (via the `RECOMMEND TO ME`
trigger). Answer through these five lenses **before** giving a verdict. Ground
every lens in evidence from this repo (cite `file:line`, a commit, a test, or a
doc section); never hand-wave. If a lens can't be determined, say so rather than
inventing it.

1. **Author intent (inferred from code).** What was the original author evidently
   trying to achieve? Infer from structure, naming, types, comments, tests, and
   git history — not from what would be nice.

2. **Least astonishment (user expectation).** Name the caller/user persona, then
   state the behaviour a reasonable such user would expect — the least
   surprising outcome given the interface, its name, and surrounding
   conventions.

3. **What the software does now.** The actual current behaviour, traced from the
   real code path. Read it; run or test it when that's cheap. This is fact, not
   intent or hope — the gaps between (1), (2), and (3) are where bugs and
   surprises live.

4. **Recommendation following the documentation.** Treating the docs (README,
   docstrings, specs, contract comments) as authoritative, what's the right
   action?

5. **Recommendation ignoring the documentation.** Setting docs aside and
   reasoning purely from code correctness, author intent, and least
```

```
   astonishment, what's the right action?

**Then synthesise — this is the point, not the five sections:**

- Call out explicitly where author intent, user expectation, and current
  behaviour **disagree**. Those gaps are the real findings.
- If (4) and (5) **diverge**, the documentation is out of step with reality. Say
  so, and recommend which to change: fix the code to match the docs, or fix the
  docs to match correct behaviour.
- End with **one** clear recommendation and the single reason it wins.

Keep each lens to a few tight sentences; spend the words on the synthesis.

---

Licensed under the [Artistic License 2.0](https://www.perlfoundation.org/artistic-license-20.html). © 2026 Steve
Bertrand.
```